

StockGuard AI Final Project Report

A Predictive Business Intelligence System for SMEs

Final Year Project 30

BS Computer Science

Muhammad Ahsan Qasim SP-2023-BSCS-041

Arslan Ahmad SP-2023-BSCS-043

Supervised by Miss Tayyaba Sultana

Department of Computer Science

Lahore Garrison University

October 2026

Contents

1. Executive summary
2. Problem and objectives
3. Scope delivered
4. System architecture
5. Forecasting methodology
6. Replenishment decision support
7. Security and administration
8. Notifications and reports
9. Testing and verification
10. Backup and recovery
11. Requirements completion
12. Limitations and future work
13. Conclusion
14. Supporting project documents

Project identification

Project: StockGuard AI - A Predictive Business Intelligence System for SMEs **FYP ID:** 30 **Programme:** BS Computer Science **Students:** Muhammad Ahsan Qasim (SP-2023-BSCS-041) and Arslan Ahmad (SP-2023-BSCS-043) **Supervisor:** Miss Tayyaba Sultana **Department:** Department of Computer Science, Lahore Garrison University **Implementation status:** Production prototype complete and deployed **Production URL:** https://stockguard-ai-ten.vercel.app

Executive summary

StockGuard AI is a multi-tenant predictive business intelligence system designed for small and medium-sized enterprises. It converts historical sales data into product-level demand forecasts, revenue estimates, inventory recommendations, operational alerts, reports and business insights. The system combines a responsive React web application, a React Native mobile application, a Node.js and Express API, MongoDB Atlas, and a Python forecasting service.

The completed prototype implements the functional requirements defined in the submitted proposal and SRS. Users can register, sign in, manage products, upload validated sales CSV files, compare forecasting models, save 7, 14, 28 or 30-day predictions, and review replenishment actions. Administrators can manage users, roles, permissions, invitations and audit logs. Production verification covers the complete application, forecasting accuracy metrics, performance under concurrent use, encrypted backup and isolated restore.

Problem and objectives

SMEs often estimate demand manually and lack affordable analytical tools. This can cause stockouts, excess inventory, lost revenue and weak financial planning. StockGuard AI addresses the problem with five practical objectives:

1. Produce demand forecasts for individual product and store series.
2. Compare Linear Regression, ARIMA, Random Forest and the available production candidate using MAE and RMSE.
3. Convert forecasts into revenue and replenishment decisions.
4. provide secure role-based access through web and mobile clients.
5. Present results in dashboards, reports, alerts and business insights that non-technical users can understand.

Scope delivered

The completed system provides:

- Email and password registration, JWT authentication, secure refresh sessions and password recovery.
- Owner, administrator, manager, analyst and staff access with configurable permissions.
- Product, store, inventory and sales management isolated by business workspace.
- CSV uploads up to 10 MB and 10,000 rows with header, date, numeric and duplicate validation.
- Data cleaning, daily-series construction and time-based feature engineering.

- Product-level forecasts for 7, 14, 28 and 30 days.
- Linear Regression, ARIMA, Random Forest, global XGBoost and seasonal-naive forecasting paths.
- Automatic model selection using lowest holdout MAE and RMSE as the tie-breaker.
- Revenue forecasting and complete daily prediction storage.
- Lead-time demand, historical variability, safety stock, target stock and recommended order quantity.
- Open, approved, dismissed and completed recommendation workflows.
- Responsive dashboards, reports, insights, notifications and administration screens.
- React Native mobile access to the same protected API and MongoDB workspace.
- Audit logging, encrypted daily backups and verified disaster recovery.

System architecture

StockGuard AI uses a layered architecture.

Layer	Technology	Responsibility
Web client	React, Vite, Recharts	Dashboard, products, uploads, forecasts, reports, insights and administration
Mobile client	React Native, Expo	Secure mobile access to operational and analytical features
Business API	Node.js, Express, Mongoose	Authentication, validation, tenant isolation, business logic and REST endpoints
Forecasting service	Python, Pandas, scikit-learn, Statsmodels	Linear Regression, Random Forest and ARIMA comparison
Production model runtime	Node.js XGBoost JSON inference	Verified global XGBoost predictions with seasonal fallback
Database	MongoDB Atlas	Users, businesses, products, sales, forecasts, reports, notifications and audit records
Hosting	Vercel and Expo Application Services	Web/API deployment and mobile build infrastructure
Operations	GitHub Actions	Encrypted daily backup, restore verification and production health monitoring

All web and mobile requests use the same REST API. Authentication and permission middleware protect application routes. Every business query includes the authenticated workspace identifier, preventing records from being shared between tenants.

Forecasting methodology

Uploaded sales records are validated and converted into continuous daily series. Missing calendar dates are represented explicitly so lag and rolling features reflect the real timeline. Forecast candidates receive a common historical window and are compared on the same holdout period.

The Python service evaluates Linear Regression, ARIMA and Random Forest. The Node.js forecasting pipeline also evaluates the verified global XGBoost model for mapped store and product identifiers; custom identifiers use a seasonal-naive candidate. The system selects the candidate with the lowest MAE, then uses RMSE to resolve a tie. It saves the selected model, metrics, forecast horizon, total demand, total revenue and every daily prediction.

The imported XGBoost production bundle was trained on 8,172,438 rows, contains 83 trees and uses 12 input features. Its three-fold recursive backtest contains 986,433 observations and records an overall MAE of 0.574939 and RMSE of 2.021802. The seasonal baseline recorded an MAE of 0.658729 and RMSE of 2.459768, giving the production model a 12.72 percent MAE improvement.

Replenishment decision support

For each completed forecast, the decision-support service calculates:

- Lead-time demand from the predicted demand during the supplier lead-time window.
- Historical demand variability from the sales series.
- Safety stock using variability, lead time and the configured safety percentage.
- Target stock from lead-time demand, safety stock and the remaining forecast period.
- Recommended order quantity after subtracting available inventory.
- Estimated days of cover and a risk classification.

The system creates low-stock, critical-stock, overstock and demand-spike recommendations. Duplicate open recommendations for the same product, store and recommendation type are prevented. Authorized users can approve, dismiss, complete or reopen an action, and high-risk actions generate notifications.

Security and administration

Passwords are stored as bcrypt hashes. Short-lived JWT access tokens and revocable refresh sessions protect web and mobile clients. Web refresh credentials use HttpOnly cookies, while the mobile client stores its refresh token in Expo SecureStore. Password changes and password resets rotate the session version so previous sessions become invalid.

Role and permission middleware restricts routes and actions. Owners and administrators can invite users, edit accounts, suspend access, assign roles, customize permissions and review system activity. Login, administration, upload, forecast, profile and security events are recorded in the audit log with one-year retention, exceeding the SRS minimum of 90 days.

Notifications and reports

The application creates persistent in-app notifications for completed uploads, completed forecasts, low stock, critical inventory and high-risk replenishment actions. Expo push registration is integrated for

mobile devices. SMTP email delivery is supported when production mail credentials are supplied; email is an optional communication integration in the SRS and does not affect in-app alerts.

Users can generate and download sales, inventory, forecast and replenishment reports. Saved reports and forecasts remain linked to the business workspace and are available to authorized web and mobile users.

Testing and verification

Verification area	Result
Backend route, model and permission assertions	Passed
Registration, login, refresh, logout and token revocation	Passed
Product CRUD, CSV idempotency and rollback	Passed
Thirty-day Python and production-model forecasting	Passed
MAE/RMSE comparison and automatic model selection	Passed
Revenue and replenishment calculations	Passed
Analytics and tenant isolation	Passed
React production build	Passed
React Native TypeScript validation and Android export	Passed
Production dependency audit	Zero known server vulnerabilities at verification time
Encrypted backup and isolated restore	Passed

The production performance test used 100 authenticated virtual users over a 10-second ramp. All 100 requests returned HTTP 200. Average response time was 259 ms, p95 was 832 ms and the maximum was 1,210 ms. Login completed in 855 ms, the dashboard in 843 ms, a live 30-day forecast in 2,224 ms and local validation of a 10,000-row CSV in 61 ms. These results meet the submitted performance requirements.

The presentation workspace contains eight products, 800 sales records, a complete 30-day forecast, three downloadable reports, operational notifications and replenishment recommendations. This dataset supports a repeatable end-to-end demonstration without modifying production business data immediately before assessment.

Backup and recovery

GitHub Actions creates an AES-256-GCM encrypted MongoDB snapshot every 24 hours. The workflow retains encrypted artifacts for 30 days and performs an isolated restore verification using temporary collection names. The verified production run restored 1,137 documents across 16 collections in 23.454 seconds and removed the temporary collections afterward. This is within the SRS recovery objective of ten minutes.

Requirements completion

All functional requirements REQ-1 through REQ-18 are mapped to implementation evidence in the SRS compliance matrix. The implemented system also meets the recorded performance, usability, scalability and recovery requirements. The web application, API, MongoDB database, Python forecasting service and production XGBoost runtime are deployed and connected.

Remaining operational configuration does not change the implemented FYP scope. SMTP credentials are required only for real outbound email, physical push receipt requires platform credentials and a device build, and long-term uptime evidence accumulates after monitoring runs over time. Automated production health monitoring and EAS mobile build profiles are included in the final repository.

Limitations and future work

- Forecast accuracy depends on the continuity and quality of uploaded sales data.
- New products with little history use conservative fallback behaviour until sufficient data becomes available.
- Email delivery requires a selected SMTP provider and credentials.
- App Store distribution requires Apple Developer approval; Android store publishing requires Play Console credentials.
- Future versions can add promotions, holidays, supplier reliability, weather and economic variables.
- Continuous MongoDB point-in-time recovery can supplement the verified encrypted daily backup when the production subscription supports it.

Conclusion

StockGuard AI delivers the prototype committed to in the proposal and SRS. It provides secure multi-user product and sales management, validated data ingestion, model comparison, product-level demand and revenue forecasting, inventory recommendations, dashboards, reports, alerts, administration and cross-platform access. The verified implementation demonstrates that an SME can move from historical sales data to a practical restocking decision in one integrated system.

Supporting project documents

- Software Requirement Specification
- Final Year Project Proposal
- SRS compliance matrix
- Final QA report
- Fifteen-minute user guide
- Presentation guide
- Backup and recovery guide
- Machine-readable performance and recovery evidence